

LC-2472: Maximum Number of Non-overlapping Palindrome Substrings

Companies: Salesforce, Walmart

Problem Overview

Given: A string S and a positive integer k .

- Length of each substring is at least k .
- Each substring is a palindrome.

Output: Max number of substrings. (Need to select a set of substrings).

Example: abaccdbbd, $k = 3$

Selected palindromes: aba and bbd

Hence: 2

Intuition & Logic

Hence, precompute if $[l][r]$ is palindrome or not.

And then just need **ab partition dp**:

$$\begin{array}{cc} [0 \dots j-1] & [j \dots i-1] \\ \text{(already processed)} & \text{(current part)} \end{array}$$

ab ya to i^{th} character is a part of palindrome, so if $[j][i-1]$ is palindrome and $i - j \geq k$, then:

$$dp[i] = \max(dp[i], dp[j] + 1)$$

Or we decide to skip characters. Option 2 is $s[i-1]$ ko use nahi kar, skip. So:

$$dp[i] = \max(dp[i], dp[i-1])$$

C++ Implementation

```
class Solution {
public:
    int maxPalindromes(string s, int k) {
        int n = s.size();
        vector<vector<bool>> ispalin(n, vector<bool>(n, false));

        for(int i = 0; i < n; i++) {
            ispalin[i][i] = true;
        }

        for(int len = 2; len <= n; len++) {
            for(int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if(s[l] == s[r]) {
                    if(len == 2) {
                        ispalin[l][r] = true;
                    } else {
                        ispalin[l][r] = ispalin[l+1][r-1];
                    }
                }
            }
        }

        // ab partition dp
        // [0.....j-1][j.....i-1]
        // ab ya to ith character is a part of palindrome
        // then dp[i]=max(dp[i],dp[j]+1)

        // or we decide to skip it
        // then dp[i]=max(dp[i],dp[i-1]);

        vector<int> dp(n+1, 0);
        for(int i = 1; i <= n; i++) {
            dp[i] = max(dp[i], dp[i-1]); // skip option
            for(int j = i - 1; j >= 0; j--) {
                if(ispalin[j][i-1] && ((i-j) >= k))
                    dp[i] = max(dp[i], dp[j]+1);
            }
        }

        return dp[n];
    }
};
```

```
}  
};
```